Dosen Pengampu:

**Dian Hanifudin Subhi, S.Kom., M.Kom.**





## Disusun oleh:

Rayhan Jofan Halim

254107020230

T1 – 2B

D-IV Teknik Informatika


Tanggal: 27 Oktober 2026


## POLITEKNIK NEGERI MALANG

Jl. Soekarno Hatta No.9, Jatimulyo, Kec. Lowokwaru, Kota Malang, Jawa Timur

65141


**TAHUN 2026-2027**

**Penjelasan Setiap Bagian 1 sampai 7**

**Bagian 1 — Pengantar**

**A. Penjelasan Konsep**

Pada bagian pertama, saya mempelajari dasar menjalankan program Java dan memahami fungsi method `main()` sebagai titik awal eksekusi program. Program dimulai dengan menampilkan teks `Welcome to Polinema Library` sesuai ketentuan pada README. Ketika program dijalankan, compiler menerjemahkan kode sumber Java menjadi bytecode, kemudian Java Virtual Machine (JVM) menjalankan bytecode tersebut. Method `main()` menggunakan kata kunci `static` agar dapat dipanggil tanpa harus membuat objek dari class `Main` terlebih dahulu.

**B. Potongan Kode dan Penjelasan**

```
1
2   package id.ac.polinema.library;
3
4   import java.util.Scanner;
5
6   public class Main {
7
        Run | Debug
8       public static void main(String[] args) {
9           System.out.println(x: "Welcome to Polinema Library");
```

Penjelasan setiap baris:

1. `public class Main` mendeklarasikan class bernama `Main` sebagai tempat program dimulai.
2. `public static void main(String[] args)` merupakan method utama yang dijalankan saat program dimulai.
3. `System.out.println()` digunakan untuk menampilkan teks ke konsol.
4. Teks di dalam tanda kutip merupakan keluaran yang harus sesuai dengan instruksi, termasuk huruf kapital dan spasinya.
5. Kurung kurawal menandai awal dan akhir blok class serta method.

**C. Jawaban Uji Pemahaman**

Lima contoh objek di perpustakaan adalah buku, DVD, majalah, anggota, dan rak. Buku memiliki data berupa judul dan ISBN serta aksi meminjam dan mengembalikan buku. DVD memiliki data judul dan durasi serta dapat dipinjam dan dikembalikan. Majalah memiliki data judul dan nomor edisi, sedangkan anggota memiliki ID dan nama serta dapat melakukan peminjaman. Rak menyimpan kumpulan buku dan dapat mencari buku berdasarkan ISBN.

**D. Kendala dan Solusi**

Kendala yang perlu diperhatikan pada bagian ini adalah memastikan teks keluaran sesuai dengan instruksi. Solusinya adalah memeriksa kembali teks pada `System.out.println()` dan menjalankan pengujian `P01MainOutputTest` untuk mengetahui apakah keluaran program sudah benar.

## Bagian 2 — Class dan Object

### A. Penjelasan Konsep

Pada bagian kedua, saya mempelajari konsep class dan object dalam Java. Class merupakan rancangan yang menentukan data dan perilaku suatu objek, sedangkan object adalah hasil pembuatan instance dari class tersebut. Dalam proyek perpustakaan, class `Author` digunakan untuk menyimpan informasi penulis, sedangkan class `Book` digunakan untuk merepresentasikan buku yang memiliki ISBN, judul, tahun, dan status ketersediaan. Constructor digunakan untuk memberikan nilai awal kepada objek, sementara getter digunakan untuk mengambil nilai yang tersimpan.

### B. Potongan Kode dan Penjelasan

Contoh penggunaan objek sesuai konsep pada bagian ini:




```
book.setAuthor(author);

System.out.println(author.getInfo());

System.out.println(book.getTitle());
```

Penjelasan setiap baris:

1. Baris pertama membuat objek `Author` dengan nama Andrea Hirata dan negara Indonesia.
2. Baris kedua membuat objek `Book` dengan ISBN, judul, dan tahun terbit.
3. Baris ketiga menghubungkan objek buku dengan objek penulis melalui method `setAuthor()`.
4. Baris keempat memanggil `getInfo()` untuk menampilkan informasi penulis.
5. Baris kelima memanggil `getTitle()` untuk mengambil judul buku.

### C. Jawaban Uji Pemahaman

Jika variabel `b` diisi dengan `a`, seperti `Book b = a`, keduanya merujuk pada objek buku yang sama. Oleh karena itu, ketika `b.checkOut()` berhasil, status ketersediaan yang dibaca melalui `a.isAvailable()` juga menjadi `false`. Sementara itu, objek `c` yang dibuat secara terpisah memiliki

status ketersediaannya sendiri. Jika constructor berhasil membuat buku `Refactoring`, status awalnya adalah tersedia, sehingga `c.isAvailable()` menghasilkan `true`.

### D. Kendala dan Solusi

Kendala pada bagian ini adalah memahami perbedaan antara class, objek, dan variabel referensi yang menunjuk ke objek. Solusinya adalah mencoba membuat beberapa objek, memberikan nilai yang berbeda, dan menjalankan method pada masing-masing objek untuk mengamati apakah perubahan pada satu objek memengaruhi objek lainnya.

## Bagian 3 — Enkapsulasi

### A. Penjelasan Konsep

Enkapsulasi merupakan konsep untuk membatasi akses langsung terhadap data objek. Dalam proyek ini, field pada class `Book` dan `Member` dibuat `private` agar tidak bisa diubah secara sembarangan dari luar class. Perubahan data dilakukan melalui setter dan method yang menyediakan validasi sesuai aturan program. Sebagai contoh, method `setName()` memeriksa nama anggota agar tidak bernilai `null`, kosong, atau hanya berisi spasi. Selain itu, jumlah pinjaman anggota harus tetap berada di antara nol dan tiga.

### B. Potongan Kode dan Penjelasan

Contoh implementasi validasi pada class `Member`:

```
67      public void setName(String name) {
68          // TODO Bantuan: periksa dulu, baru isi field.
69          //
70          // Kerangka:
71          // if (name == null || ____) {
72          // throw new IllegalArgumentException("____");
73          // }
74          // this.name = name;
75          if (name == null || name.trim().isEmpty()) {
76              throw new IllegalArgumentException(s: "Name cannot be null or empty");
77          }
78          this.name = name;
79      }
```

```
87      public boolean canBorrow() {
88          // TODO Bantuan: apakah perlu field baru, atau cukup dihitung dari data yang
89          // sudah ada?
90          //
91          // Kerangka:
92          // return loanCount ____ 3;
93          return loanCount < 3;
94      }
```

Penjelasan setiap baris:

1. `setName(String name)` merupakan setter untuk mengubah nama anggota.
2. Kondisi `name == null` memeriksa apakah nama tidak memiliki nilai.
3. `name.trim().isEmpty()` memeriksa apakah nama kosong atau hanya berisi spasi.
4. `throw new IllegalArgumentException(...)` menolak nilai yang tidak sesuai aturan.
5. `this.name = name` menyimpan nama baru hanya jika validasi berhasil.

6. `canBorrow()` digunakan untuk menentukan apakah anggota masih boleh meminjam.
7. Kondisi `loanCount < 3` memastikan anggota hanya boleh memiliki maksimal tiga pinjaman.

### C. Jawaban Uji Pemahaman

Jika `loanCount` dibuat `public`, kode dari luar class dapat mengubahnya secara langsung, misalnya menjadi 100, sehingga aturan maksimal tiga pinjaman dapat dilanggar. Setelah field dibuat `private`, akses langsung seperti itu tidak dapat dilakukan dari class lain. Constructor menggunakan validasi yang sama dengan setter agar objek tidak dapat dibuat dengan data yang tidak valid sejak awal. Method `canBorrow()` tidak memerlukan field tambahan karena hasilnya dapat dihitung langsung berdasarkan `loanCount`. Sementara itu, `memberId` tidak perlu memiliki setter karena identitas anggota sebaiknya tetap setelah objek dibuat.

### D. Kendala dan Solusi

Kendala pada bagian ini adalah memastikan validasi tidak mengubah data lama ketika nilai baru tidak valid. Solusinya adalah memeriksa nilai terlebih dahulu, baru menyimpan nilai baru jika seluruh pemeriksaan berhasil. Saya juga perlu memastikan field yang bersifat read-only tidak memiliki setter dan semua method mengikuti batasan yang telah ditentukan.

## Bagian 4 — Relasi Class

### A. Penjelasan Konsep

Pada bagian keempat, saya mempelajari empat jenis relasi class, yaitu association, composition, aggregation, dan dependency. Association terjadi antara `Book` dan `Author` karena sebuah buku dapat menyimpan referensi ke objek penulis. Composition terdapat pada hubungan `Member` dan `LibraryCard`, karena setiap anggota memiliki kartu yang dibuat sebagai bagian dari objek anggota. Aggregation terjadi antara `Shelf` dan `Book`, karena rak menyimpan referensi terhadap buku yang dibuat dari luar. Dependency terdapat pada class `Librarian` yang menggunakan objek `Member` dan `Book` melalui parameter method tanpa menyimpannya sebagai field.

### B. Potongan Kode dan Penjelasan

Contoh relasi dependency pada class `Librarian`:

```
 9    public boolean lend(Member member, Book book) {
10        if (member == null || book == null) {
11            return false;
12        }
13        // Syarat: anggota boleh meminjam DAN buku sedang tersedia
14        if (member.canBorrow() && book.isAvailable()) {
15            book.checkOut();
16            member.addLoan();
17            return true;
18        }
19        return false;
20    }
```

Penjelasan setiap baris:

1. `lend(Member member, Book book)` menerima objek anggota dan buku melalui parameter.
2. Pemeriksaan `null` memastikan kedua objek tersedia sebelum digunakan.
3. `canBorrow()` dan `isAvailable()` memeriksa kelayakan peminjaman.
4. `book.checkOut()` mengubah status buku menjadi dipinjam jika masih tersedia.
5. `member.addLoan()` menambah jumlah pinjaman anggota.
6. Jika penambahan pinjaman gagal, `book.returnItem()` mengembalikan status buku agar tidak terjadi perubahan yang tidak konsisten.
7. `return true` menunjukkan bahwa proses peminjaman berhasil.

**C. Jawaban Uji Pemahaman**

Jika objek `Member` tidak lagi digunakan, kartu `LibraryCard` yang menjadi bagian dari objek tersebut tidak lagi memiliki referensi dari anggota itu. Jika objek `Shelf` tidak digunakan lagi, objek `Book` yang pernah disimpan di dalamnya tetap dapat digunakan jika masih dirujuk oleh bagian program lain. Perbedaan ini menjelaskan bahwa composition menunjukkan kepemilikan bagian yang lebih kuat, sedangkan aggregation menunjukkan hubungan penyimpanan atau pengelompokan objek tanpa mengharuskan siklus hidupnya sama.

**D. Kendala dan Solusi**

Kendala pada bagian ini adalah membedakan aggregation dan composition karena keduanya sama-sama dapat melibatkan penyimpanan referensi objek. Solusinya adalah memeriksa siapa yang membuat objek, bagaimana objek diberikan kepada class lain, serta apakah objek tersebut dibuat sebagai bagian dari pemiliknya atau berasal dari luar. Saya juga perlu memastikan class `Librarian` tidak menyimpan field karena sesuai rancangan, class tersebut hanya menggunakan objek melalui parameter method.

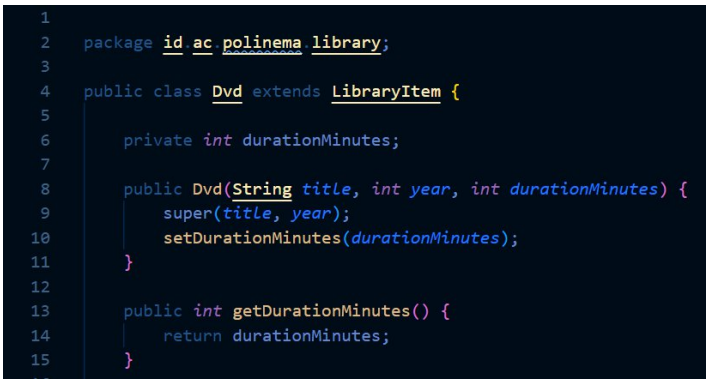
**Bagian 5 — Inheritance**

**A. Penjelasan Konsep**

Inheritance atau pewarisan memungkinkan sebuah class memperoleh atribut dan method dari class induknya. Pada proyek perpustakaan, `Book`, `Dvd`, dan `Magazine` merupakan subclass langsung dari `LibraryItem`. Class `LibraryItem` menyimpan data umum berupa judul, tahun terbit, dan status ketersediaan, sedangkan subclass menyimpan data khusus sesuai jenis koleksi. Field `title` dan `year` menggunakan modifier `protected` agar dapat diakses oleh subclass, sedangkan data lainnya tetap dibatasi sesuai kebutuhan. Penggunaan inheritance mengurangi duplikasi kode karena perilaku umum cukup ditulis di class induk.

**B. Potongan Kode dan Penjelasan**

Contoh inheritance pada class `Dvd`:



```java

package id.ac.polinema.library;

public class Dvd extends LibraryItem {

    private int durationMinutes;

    public Dvd(String title, int year, int durationMinutes) {
        super(title, year);
        setDurationMinutes(durationMinutes);
    }

    public int getDurationMinutes() {
        return durationMinutes;
    }
```

Penjelasan setiap baris:

1. `public class Dvd extends LibraryItem` menyatakan bahwa `Dvd` merupakan subclass dari `LibraryItem`.
2. `durationMinutes` menyimpan durasi DVD dan hanya dapat diakses langsung dari dalam class.
3. Constructor `Dvd` menerima judul, tahun, dan durasi.
4. `super(title, year)` memanggil constructor parent untuk menginisialisasi data umum.
5. `setDurationMinutes(durationMinutes)` mengisi durasi dengan menerapkan validasi yang disediakan.
6. `getDurationMinutes()` mengembalikan nilai durasi DVD.

### C. Jawaban Uji Pemahaman

Class `Dvd` mewarisi atribut dan method umum dari `LibraryItem`, seperti `getTitle()`, `getYear()`, `isAvailable()`, `checkOut()`, dan `returnItem()`. Constructor `LibraryItem` tidak diwarisi oleh `Dvd`, sehingga constructor subclass harus memanggilnya melalui `super(title, year)`. Pemanggilan `super(...)` harus menjadi pernyataan pertama pada constructor subclass. Array bertipe `LibraryItem[]` dapat menampung objek `Book`, `Dvd`, dan `Magazine` karena ketiganya merupakan subclass dari `LibraryItem`.

### D. Kendala dan Solusi

Kendala yang saya temui pada bagian ini adalah ketidaksesuaian nama getter durasi DVD dengan diagram dan tes otomatis. Tes mengharapkan `getDurationMinutes()`, sedangkan implementasi awal menggunakan `getDuration()`. Solusinya adalah menyesuaikan nama field dan getter dengan diagram, kemudian menjalankan kembali `P06InheritanceTest` untuk memastikan kesalahan tersebut telah diperbaiki.

## Bagian 6 — Overriding dan Overloading

### A. Penjelasan Konsep

Pada bagian keenam, saya mempelajari overriding dan overloading. Overriding terjadi ketika subclass memberikan implementasi baru untuk method yang diwarisi dari parent, sedangkan overloading terjadi ketika beberapa method memiliki nama sama tetapi daftar parameternya berbeda. Dalam proyek ini,

`Book` meng-override `loanDays()` menjadi 14 hari dan `Dvd` meng-override `loanDays()` menjadi tiga hari. `Magazine` tetap menggunakan method `loanDays()` dari `LibraryItem`, sehingga lama pinjamnya tujuh hari. Overloading diterapkan pada `extendLoan()` dan `extendLoan(int days)` untuk memberikan dua cara memperpanjang masa pinjam.

**B. Potongan Kode dan Penjelasan**

Contoh overloading pada class `LibraryItem`:



```
77      public void extendLoan() {
78          extendLoan(days: 7);
79      }
80
81      public void extendLoan(int days) {
82          if (days <= 0) {
83              throw new IllegalArgumentException(
84                  s: "Extension days must be greater than zero"
85              );
86          }
87
88          extensionDays += days;
89      }
90
91      public int getTotalLoanDays() {
92          return loanDays() + extensionDays;
93      }
94  }
95
```

Penjelasan setiap baris:

1. `extendLoan()` merupakan method tanpa parameter untuk memperpanjang masa pinjam dengan nilai standar.
2. `extendLoan(7)` memanggil method overload dengan nilai tujuh hari.
3. `extendLoan(int days)` menerima jumlah hari sesuai parameter.
4. Pemeriksaan `days <= 0` menolak penambahan hari yang tidak valid.
5. `IllegalArgumentException` digunakan untuk menandai input yang tidak sesuai aturan.
6. `extensionDays += days` menambahkan jumlah hari perpanjangan yang tersimpan.
7. `getTotalLoanDays()` menghitung total hari pinjam dengan menjumlahkan lama pinjam standar dan seluruh perpanjangan.
8. Pemanggilan `loanDays()` memungkinkan lama pinjam standar mengikuti implementasi subclass.

**C. Jawaban Uji Pemahaman**

Overriding digunakan ketika subclass membutuhkan perilaku yang berbeda dari parent, seperti `loanDays()` pada `Book` dan `Dvd`. Overloading digunakan ketika suatu operasi memiliki beberapa bentuk pemanggilan, seperti `extendLoan()` tanpa parameter dan `extendLoan(int days)` dengan parameter. Anotasi `@Override` membantu compiler memastikan bahwa method yang ditulis benar-benar meng-override method dari parent. Jika nama atau parameter method salah, compiler akan menampilkan error bahwa method tersebut tidak meng-override atau mengimplementasikan method dari supertype.

### D. Kendala dan Solusi

Kendala yang perlu diperhatikan adalah memastikan method yang dioverride memiliki nama dan parameter yang sesuai dengan class induk. Selain itu, validasi nilai perpanjangan diperlukan agar jumlah hari yang tidak valid tidak disimpan. Solusinya adalah menggunakan anotasi `@Override`, memanfaatkan `super.describe()` pada subclass, serta menjalankan `P07OverrideTest` dan `P07OverloadTest` untuk memeriksa kesesuaian implementasi.

## Bagian 7 — Aplikasi Konsol

### A. Penjelasan Konsep

Pada bagian ketujuh, saya menggabungkan class-class yang telah dibuat sebelumnya menjadi aplikasi konsol perpustakaan. Class `Library` bertanggung jawab mengelola koleksi dan anggota, termasuk menambah data, mencari koleksi, meminjam, serta menerima pengembalian. Class `LibraryApp` bertanggung jawab menampilkan menu, membaca input pengguna, dan memberikan pesan hasil, sedangkan class `Main` menjadi titik awal program yang mengisi data contoh dan menjalankan aplikasi. Pemisahan tanggung jawab tersebut membuat logika bisnis tidak bercampur dengan tampilan konsol dan memudahkan pengujian menggunakan `Scanner` serta `PrintStream`.

### B. Potongan Kode dan Penjelasan

Contoh alur peminjaman pada class `Library`:

```
106     public boolean lend(String memberId, String title) {
107         Member member = findMember(memberId);
108         LibraryItem item = findItemByTitle(title);
109 
110         if (member == null
111                 || item == null
112                 || !member.canBorrow()
113                 || !item.isAvailable()) {
114             return false;
115         }
116 
117         if (!item.checkOut()) {
118             return false;
119         }
120 
121         if (!member.addLoan()) {
122             item.returnItem();
123             return false;
124         }
125 
126         return true;
127     }
```

Penjelasan setiap baris:

1. `lend(String memberId, String title)` menerima ID anggota dan judul koleksi yang akan dipinjam.
2. `findMember(memberId)` mencari anggota berdasarkan ID.
3. `findItemByTitle(title)` mencari koleksi berdasarkan judul.
4. Pemeriksaan `member == null || item == null` menolak transaksi jika anggota atau koleksi tidak ditemukan.

5. `member.canBorrow()` dan `item.isAvailable()` memastikan anggota masih boleh meminjam dan koleksi tersedia.
6. `item.checkOut()` mengubah status koleksi menjadi dipinjam jika proses berhasil.
7. `member.addLoan()` menambah jumlah pinjaman anggota.
8. Jika penambahan pinjaman gagal, `item.returnItem()` mengembalikan status koleksi agar data tetap konsisten.
9. `return true` menandakan peminjaman berhasil.

**C. Jawaban Uji Pemahaman**

Pada alur peminjaman, `LibraryApp` membaca pilihan pengguna dan menerima ID anggota serta judul koleksi. Setelah itu, `LibraryApp` memanggil method `lend()` pada `Library`. Class `Library` mencari objek `Member` dan `LibraryItem`, memeriksa apakah anggota boleh meminjam dan koleksi tersedia, lalu memanggil `checkOut()` pada koleksi dan `addLoan()` pada anggota. Setelah transaksi selesai, `LibraryApp` menampilkan pesan `Loan successful` atau `Loan failed` berdasarkan hasil method `lend()`.

Class `Library` tidak mencetak output secara langsung karena tugasnya adalah menjalankan logika bisnis. Dengan pemisahan ini, tampilan dapat diubah tanpa harus mengubah aturan peminjaman di `Library`.

**D. Kendala dan Solusi**

Kendala pada bagian ini adalah menghubungkan input dari `LibraryApp` dengan logika transaksi pada `Library`, sekaligus memastikan status koleksi dan jumlah pinjaman anggota selalu konsisten. Solusinya adalah memisahkan proses pembacaan input, pemeriksaan transaksi, perubahan status, dan pencetakan hasil ke class yang memiliki tanggung jawab masing-masing. Pengujian `P08LibraryTest` dan `P08LibraryAppTest` perlu dijalankan untuk memeriksa logika bisnis dan keluaran aplikasi sesuai instruksi README.

**3. Tambahan Bagian 7 — Bukti Aplikasi Konsol**

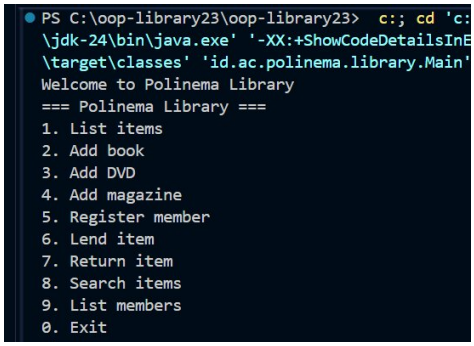
Pada Bagian 7, aplikasi konsol dijalankan untuk menunjukkan proses pengelolaan perpustakaan secara langsung. Aplikasi menyediakan menu untuk menampilkan koleksi, menambahkan buku, DVD, dan majalah, mendaftarkan anggota, melakukan peminjaman, menerima pengembalian, mencari koleksi, dan menampilkan daftar anggota.

Dalam alur peminjaman, pengguna memilih menu nomor 6 dan memasukkan ID anggota serta judul koleksi. `LibraryApp` meneruskan permintaan ke `Library`, lalu `Library` memeriksa keberadaan anggota, keberadaan koleksi, batas pinjaman anggota, dan status ketersediaan koleksi. Jika seluruh syarat terpenuhi, koleksi ditandai sedang dipinjam dan jumlah pinjaman anggota bertambah. Setelah itu, aplikasi menampilkan pesan keberhasilan.

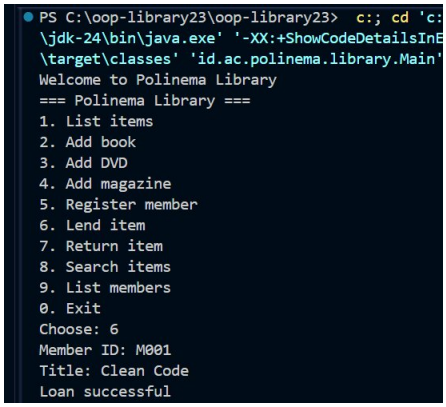
Ketika pengembalian dilakukan melalui menu nomor 7, aplikasi memeriksa transaksi melalui method `receive()`. Jika koleksi memang sedang dipinjam dan anggota memiliki pinjaman, status koleksi dikembalikan menjadi tersedia dan jumlah pinjaman anggota berkurang.

**Bukti screenshot yang dimasukkan ke laporan:**

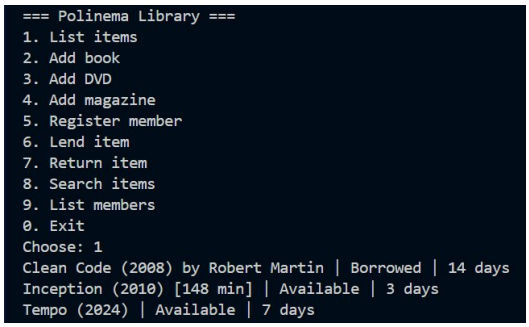
- **Gambar 1. Tampilan menu utama:** menunjukkan pilihan menu aplikasi perpustakaan.



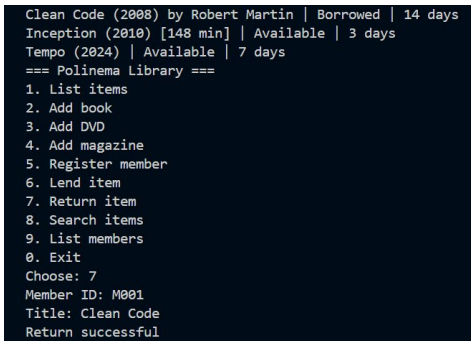

- **Gambar 2. Peminjaman koleksi:** menunjukkan pesan `Loan successful`.



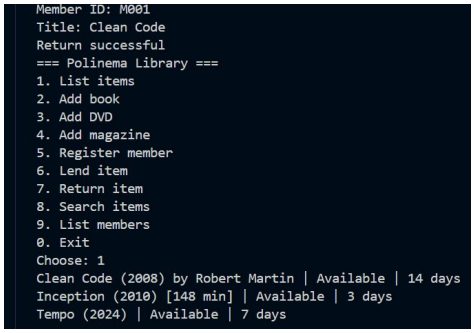

- **Gambar 3. Status koleksi setelah dipinjam:** menunjukkan koleksi berstatus `Borrowed` dan jumlah pinjaman anggota bertambah.




- **Gambar 4. Pengembalian koleksi:** menunjukkan pesan `Return successful`.




- **Gambar 5. Status setelah pengembalian:** menunjukkan koleksi kembali `Available` dan jumlah pinjaman anggota berkurang.




**6. Refleksi**

Selama mengerjakan proyek OOP Aplikasi Konsol Perpustakaan, saya mempelajari penerapan konsep pemrograman berorientasi objek mulai dari class dan object, enkapsulasi, relasi antarkelas, inheritance, overriding, overloading, hingga penggabungan beberapa class dalam aplikasi konsol. Saya memahami bahwa setiap class perlu memiliki tanggung jawab yang jelas agar program lebih terstruktur dan mudah dipelihara.

Bagian yang masih perlu saya pelajari lebih lanjut adalah alur interaksi antarclass, terutama ketika aplikasi memproses peminjaman dan pengembalian koleksi. Saya perlu memastikan bahwa objek yang tepat dipanggil, status ketersediaan koleksi berubah dengan benar, dan jumlah pinjaman anggota selalu konsisten. Saya juga perlu memperdalam pemahaman mengenai inheritance dan overriding agar dapat menjelaskan mengapa method dari subclass dapat menghasilkan perilaku yang berbeda meskipun dipanggil melalui referensi bertipe parent.

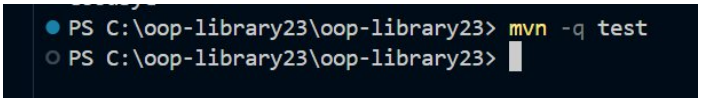
Sebagai rencana pembelajaran, saya akan mempelajari kembali kode setiap class secara bertahap, mulai dari `LibraryItem`, `Book`, `Dvd`, dan `Magazine`, kemudian melanjutkan ke `Library`, `LibraryApp`, dan `Main`. Saya akan berlatih membuat program sederhana dengan array, perulangan, percabangan, validasi input, serta method yang menghubungkan beberapa objek. Selain itu, saya akan menjalankan pengujian setelah setiap perubahan, mempelajari pesan error yang muncul, dan mencoba menjelaskan fungsi setiap method menggunakan kata-kata sendiri. Dengan latihan tersebut, saya berharap dapat membuat dan mengembangkan aplikasi Java secara lebih mandiri serta memahami hubungan antara desain class dan perilaku program.

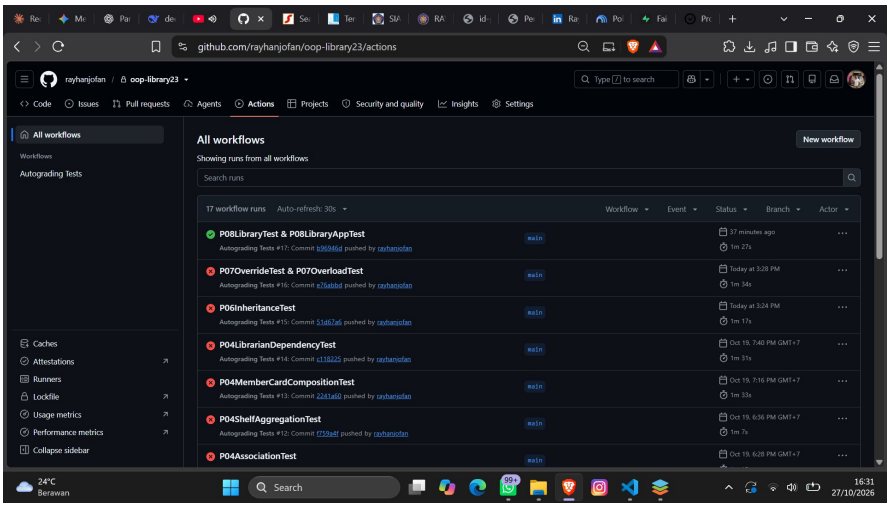
## Laporan

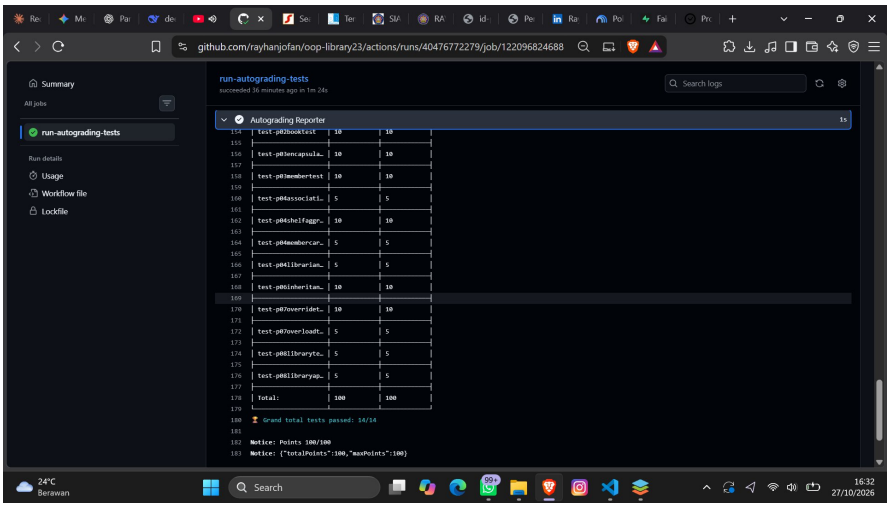
### 1. Link Github

https://github.com/rayhanjofan/oop-library23

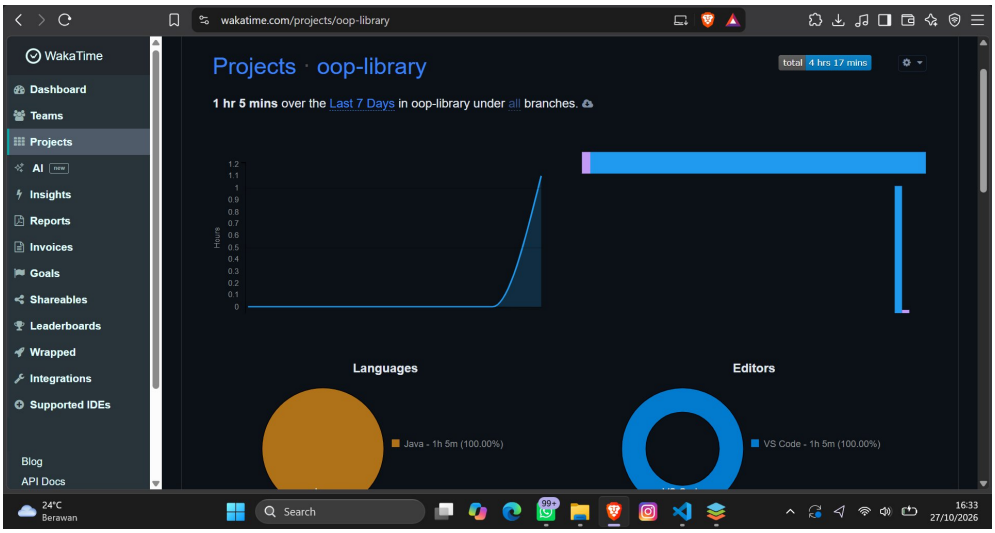
**2. Hasil pengujian**: screenshot mvn test atau tab Actions yang menampilkan nilai.




**3. Bukti WakaTime**: screenshot dashboard project uts-c-kafe dan total jam dalam teks.




### 6. Refleksi

Selama mengerjakan proyek OOP Aplikasi Konsol Perpustakaan, saya mempelajari penerapan konsep pemrograman berorientasi objek mulai dari class dan object, enkapsulasi, relasi antarkelas, inheritance, overriding, overloading, hingga penggabungan beberapa class dalam aplikasi konsol. Saya memahami bahwa setiap class perlu memiliki tanggung jawab yang jelas agar program lebih terstruktur dan mudah dipelihara.

Bagian yang masih perlu saya pelajari lebih lanjut adalah alur interaksi antarclass, terutama ketika aplikasi memproses peminjaman dan pengembalian koleksi. Saya perlu memastikan bahwa objek yang tepat dipanggil, status ketersediaan koleksi berubah dengan benar, dan jumlah pinjaman anggota selalu konsisten. Saya juga perlu memperdalam pemahaman mengenai inheritance dan overriding agar dapat menjelaskan mengapa method dari subclass dapat menghasilkan perilaku yang berbeda meskipun dipanggil melalui referensi bertipe parent.

Sebagai rencana pembelajaran, saya akan mempelajari kembali kode setiap class secara bertahap, mulai dari `LibraryItem`, `Book`, `Dvd`, dan `Magazine`, kemudian melanjutkan ke `Library`, `LibraryApp`, dan `Main`. Saya akan berlatih membuat program sederhana dengan array, perulangan, percabangan, validasi input, serta method yang menghubungkan beberapa objek. Selain itu, saya akan menjalankan pengujian setelah setiap perubahan, mempelajari pesan error yang muncul, dan mencoba menjelaskan fungsi setiap method menggunakan kata-kata sendiri. Dengan latihan tersebut, saya berharap dapat membuat dan mengembangkan aplikasi Java secara lebih mandiri serta memahami hubungan antara desain class dan perilaku program.